FORMADORES { IT }

Kubernetes

8. Seguridad

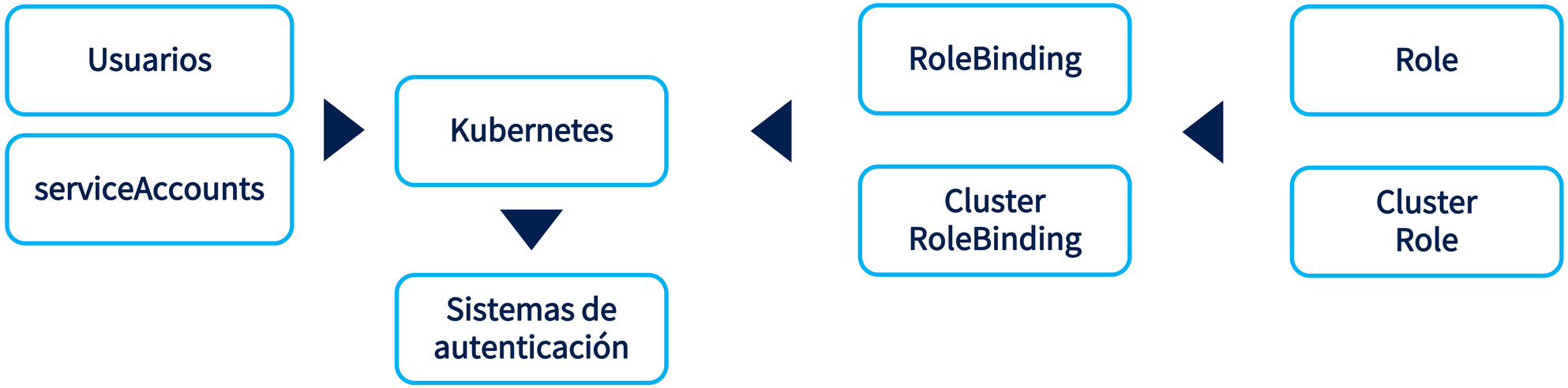
Índice

- Introducción a la seguridad en Kubernetes
- Métodos de autenticación de usuarios
- ServiceAccounts
- Métodos de autorización
- RBAC
  - Roles
  - Bindings

Objetivos

- Tener una visión amplia de los sistemas de autenticación disponibles
- Ver cómo utilizar serviceAccounts en el clúster
- Saber crear distintos roles con distintos permisos
- Asignar esos roles a usuarios, grupos y serviceAccounts

Mapa Conceptual



Seguridad en Kubernetes

Hasta ahora no hemos tenido límites para realizar cualquier acción en Kubernetes. Hemos actuado como administrador del clúster y hemos tenido acceso a todas las operaciones.

Pero Kubernetes no está pensado para que todos los usuarios de la plataforma sean administradores. De hecho tiene un sistema muy avanzado de permisos para que cada usuario sólo tenga acceso a los recursos y Namespaces que necesita.

Es más, todas las cargas de trabajo que se ejecutan en Kubernetes también pueden tener esos permisos, como si fueran un usuario más de la plataforma. Así se puede proteger al clúster ante cargas de trabajo no fiables.

Seguridad en Kubernetes

La seguridad de cualquier sistema tiene dos aspectos fundamentales:

1. Autenticación. Cómo identificar al usuario que está realizando una acción.
2. Autorización. Dar permisos concretos a cada usuario, de forma que no pueda realizar acciones fuera de ellos.

Autenticación

El usuario con el que te conectas al clúster lo marca el archivo kubeconfig.

En el apartado users se pueden configurar los usuarios de ese cliente (para el mismo clúster o para distintos clústeres).

En el clúster están configurados los sistemas de autenticación permitidos. Cada proveedor de Kubernetes tiene su sistema, y si despliegas tu propia plataforma puedes elegir entre varios disponibles.

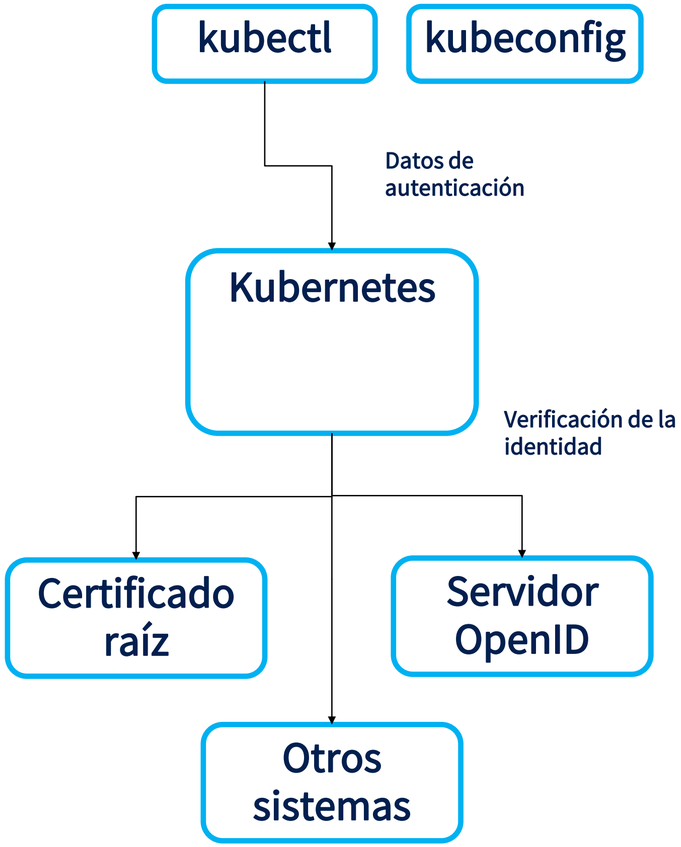
Cada sistema de autenticación tiene sus propios parámetros para la configuración en kubeconfig.

Autenticación

Los métodos de autenticación soportados por Kubernetes son:

- **Certificados X509**: Es el que usamos en el entorno de prueba y el más sencillo de implementar. Kubernetes verifica el certificado de cada cliente contra un certificado raiz.

- **Tokens o passwords estáticos**: En un fichero se pueden definir todas las credenciales válidas para usuarios. No es práctico ya que para añadir o eliminar usuarios es necesario reiniciar el clúster

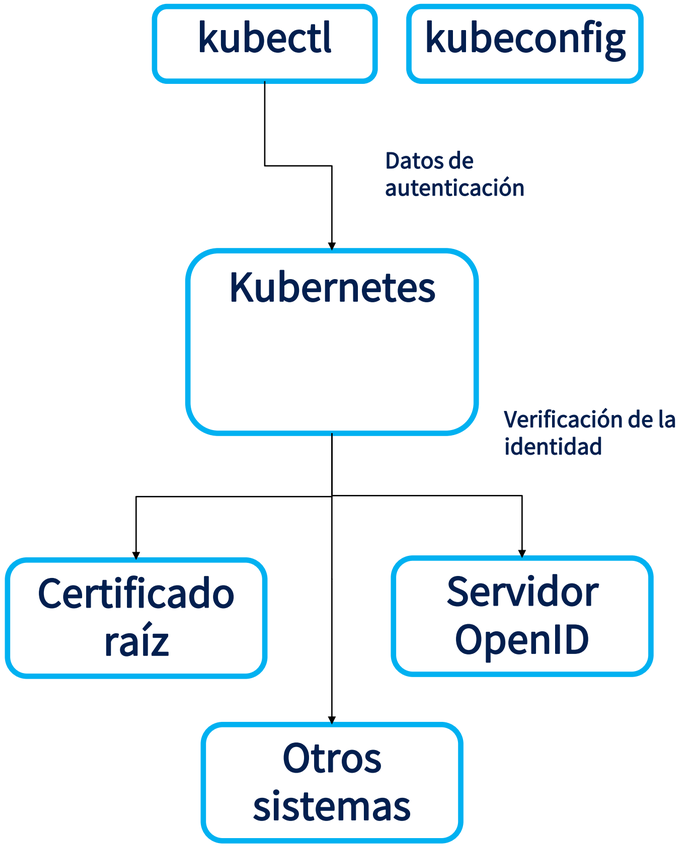


Autenticación

OpenID: Es un estándar de autenticación web, similar a OAuth2. Es soportado por la mayoría de sistemas de autenticación como Google, Facebook, etc. Con él permitimos que el servidor externo de OpenID verifique la identidad de los usuarios.

External Authentication: Kubernetes tiene un protocolo para que cualquier sistema externo pueda verificar los datos de autenticación del usuario, similar a OpenID.

Recordemos que cada clúster tiene sus métodos de autenticación, no todos están implementados en todos los clúster de Kubernetes.

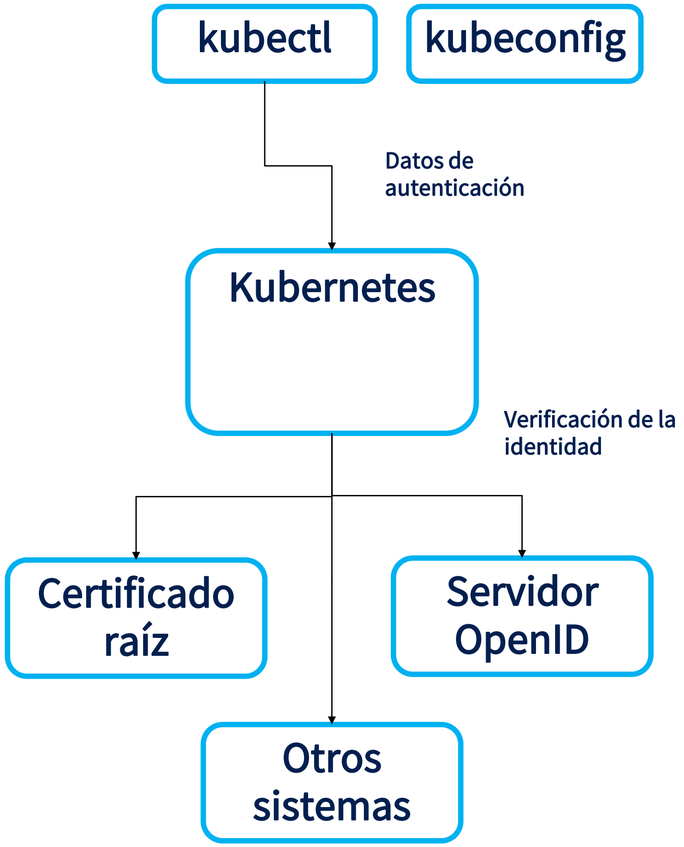


Autenticación

Normalmente el proveedor o la distribución de Kubernetes que implementes te dará las instrucciones para crear el kubeconfig, dependiendo del método seleccionado.

Por ejemplo para la autenticación por certificados, el user dentro del kubeconfig tendrá la estructura:

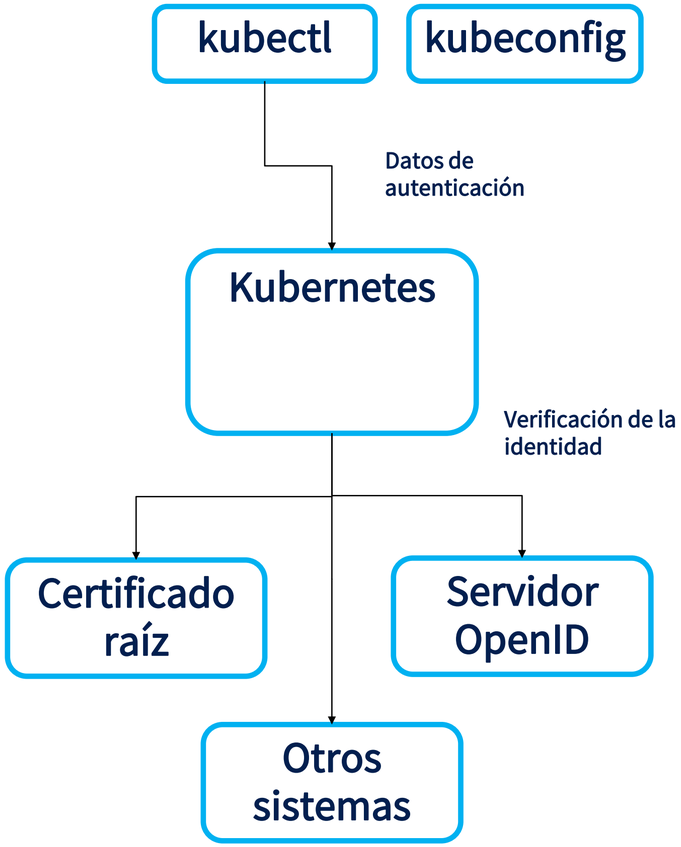
```
user:
  client-certificate-data: <certificado en base64>
  client-key-data: <certificado en base64>
```



Autenticación

Para openid:

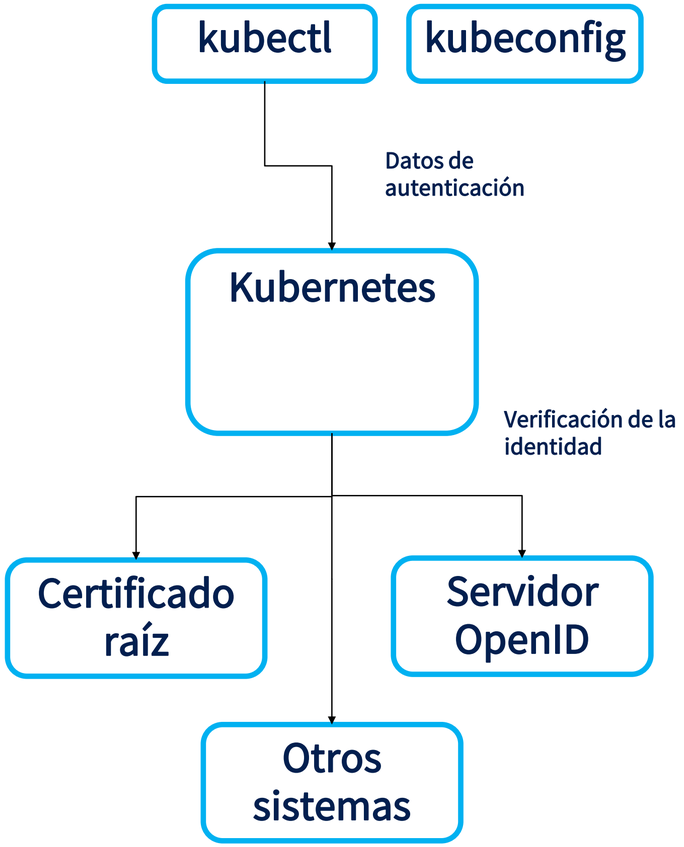
```
user:
  auth-provider:
    config:
      client-id:
      client-secret:
      id-token:
      idp-certificate-authority:
      idp-issuer-url:
      refresh-token:
```



Autenticación

Y también es posible configurar kubectl para que ejecute un programa externo para obtener las credenciales:

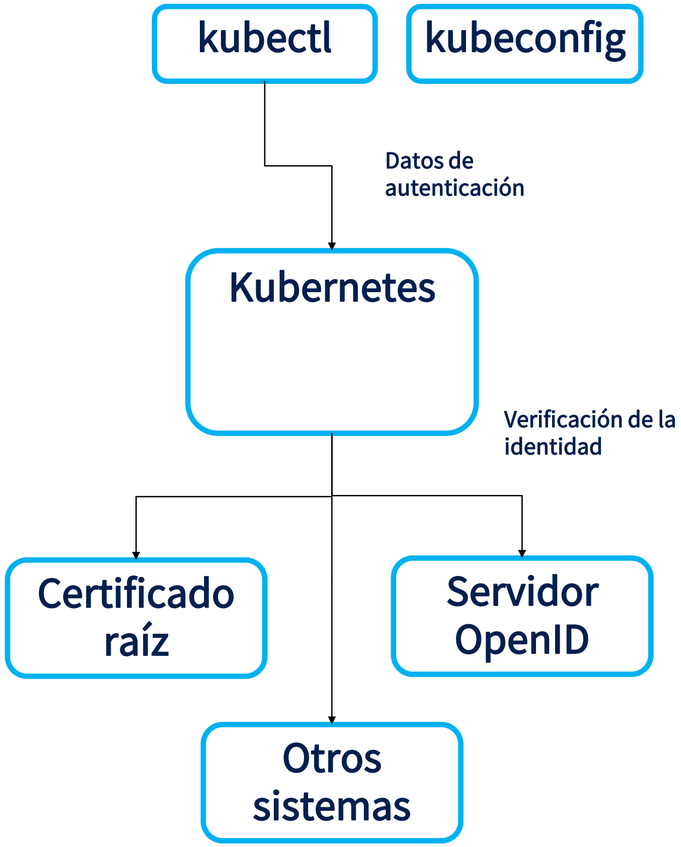
```
user:
  exec:
    apiVersion: client.authentication.k8s.io/v1alpha1
    args: <argumentos para el binario>
    command: <path al binario>
    env: null
```



Autenticación

En el entorno de prueba podemos ver que está configurado con el certificado y la clave de cliente. Este certificado ha sido emitido a partir de un certificado raíz al que tiene acceso Kubernetes para verificar que la identidad que marca el certificado es verdadera.

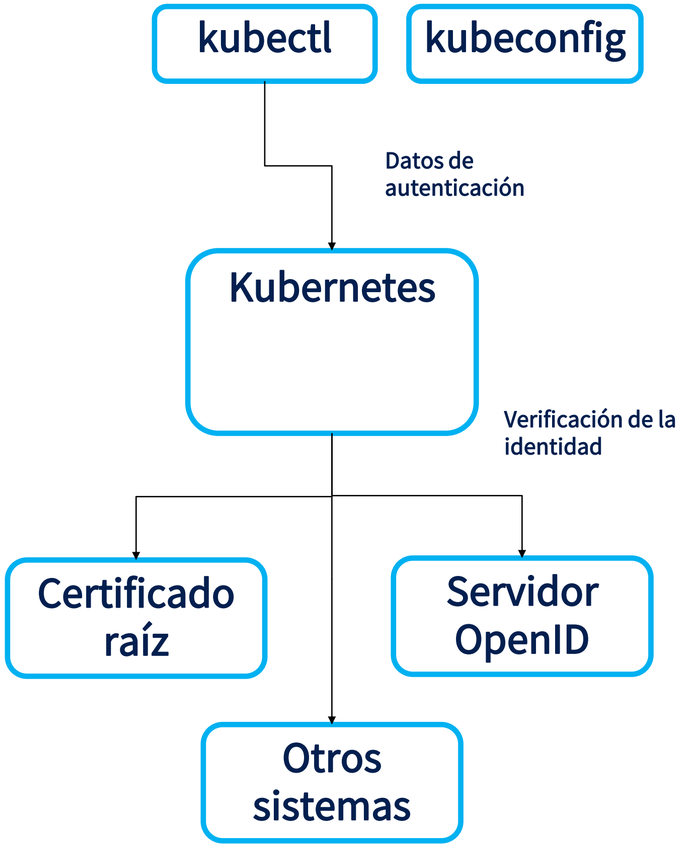
Todos los métodos de autenticación te darán un nombre de usuario y opcionalmente uno o varios grupos a los que pertenece tu usuario, verificado todo por el sistema de autenticación elegido. En el caso de los certificados, estos contienen campos para el usuario y los grupos. En OpenID, el servidor verificará tu nombre de usuario y los grupos a los que perteneces. Etc.



Autenticación

Más adelante utilizaremos el nombre de usuario o el grupo para dar permisos.

Es importante recalcar que Kubernetes no mantiene una lista de usuarios ni grupos. No es posible crear un usuario o un grupo nuevo en Kubernetes. Ese trabajo lo delega siempre en sistemas externos como certificados, servidores OpenID, ficheros con listas cerradas, etc.

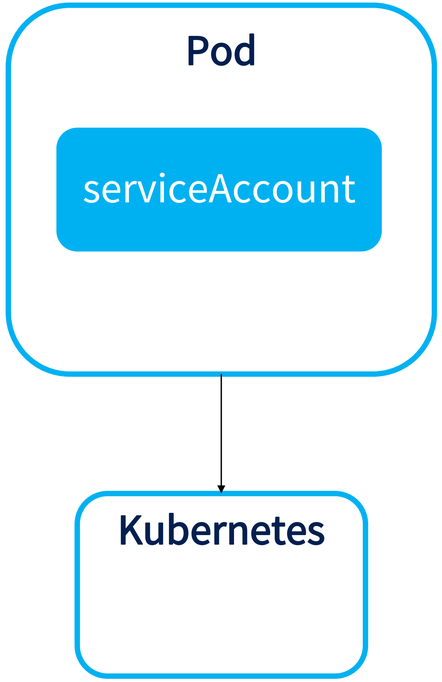


ServiceAccounts

En Kubernetes no tendremos sólo usuarios 'humanos', sino que cualquier proceso en cualquier pod puede conectarse con el clúster de Kubernetes para obtener información o realizar acciones. Esto hace que automatizar todo tipo de procesos sea de lo más sencillo.

Por ejemplo, el servicio de DNS de Kubernetes, o el mismo kube-proxy, necesitan desde sus pods conectarse al clúster para recibir la lista de services que hay desplegados, y crear las entradas DNS y de iptables para cada uno de ellos.

De la misma forma existen multitud de posibilidades, desde autoescalado de pods hasta emisión de certificados TLS de forma automática. Todo ello es posible gracias a que los pod pueden conectarse con Kubernetes fácilmente.



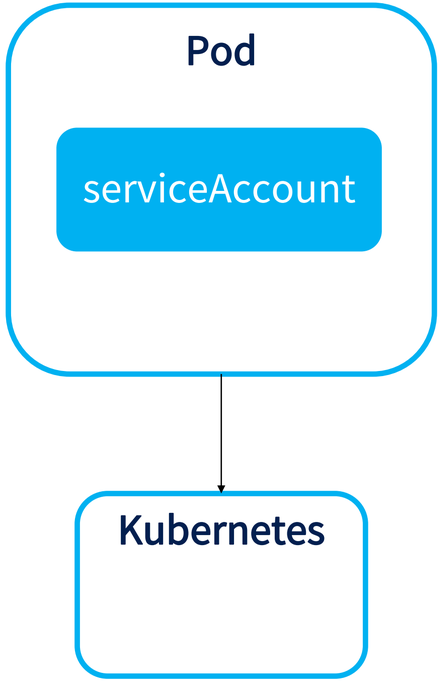
ServiceAccounts

¿Cómo se autentican los procesos dentro de los pods?

No sería práctico tener que configurar manualmente credenciales para todos los pods que las necesiten. Kubernetes inserta automáticamente tokens de autenticación en todos los pod para que éstos los puedan utilizar si lo necesitan.

Estos token son las llamadas **serviceAccounts**

Kubernetes tiene una serviceAccount por defecto que inserta en todos los pod. Nosotros podemos crear nuevas serviceAccounts para dar permisos distintos a nuevos pods.



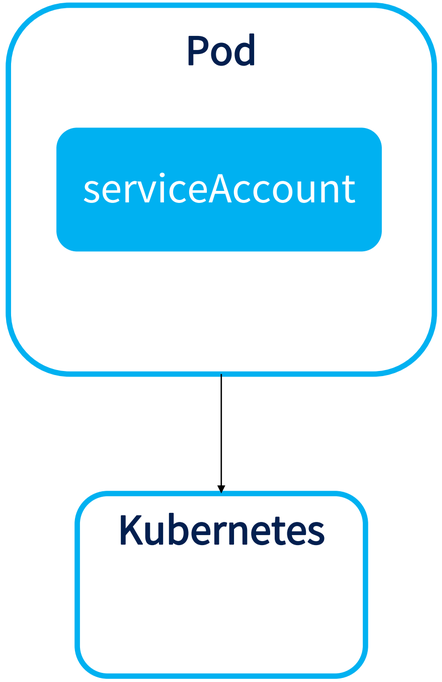
ServiceAccounts

Para ver la serviceAccount por defecto, puedes listar las del namespace default con:

```
kubectl get serviceaccounts
```

Para inspeccionar la serviceAccount por defecto de ese namespace:

```
kubectl describe serviceaccount default
```



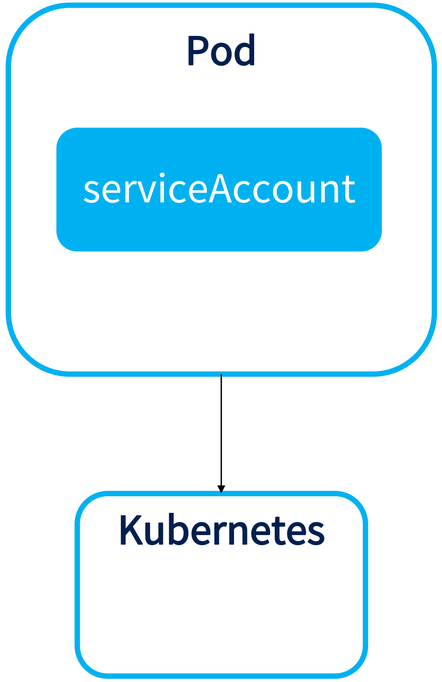
ServiceAccounts

Si creamos un pod nuevo con una terminal:

```
kubectl run --rm -it ubuntu --image ubuntu --restart=Never
```

Veremos que en la ruta /run/secrets/kubernetes.io/serviceaccount **están montados todos estos elementos.**

Esa ruta es fija para todos los pod, y cualquier método de comunicación con el clúster buscará credenciales ahí automáticamente.

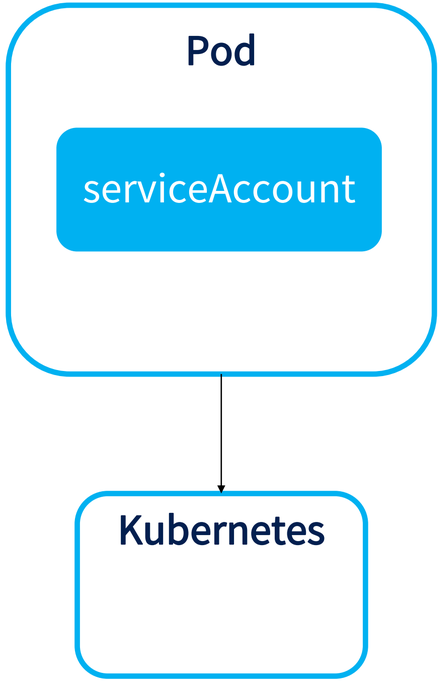


ServiceAccounts

Podemos crear una service account nueva:

```
kubectl create serviceaccount my-credentials
```

Veremos que ha creado una serviceAccount nueva y un secret asociado a ella.

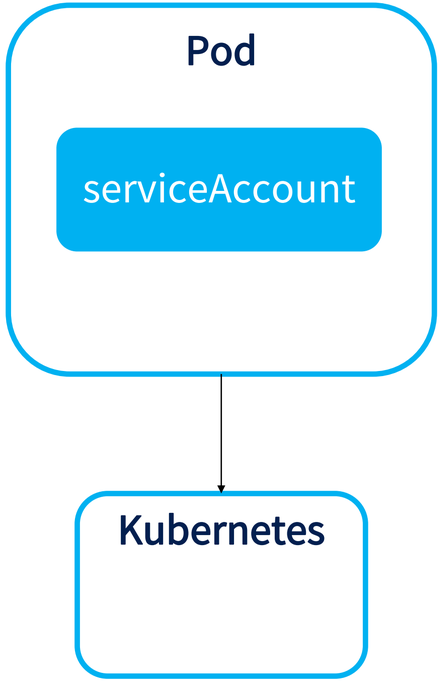


ServiceAccounts

Podemos configurar nuestros pod para que utilicen esta serviceAccount en vez de la de por defecto, añadiendo este campo en la spec del pod:

serviceAccountName: my-credentials

Es una buena práctica no dar permisos a las serivceAccount por defecto de cada namespace, sino crear nuevas serviceAccount específicas para cada pod y asignar los permisos mínimos que requiera.

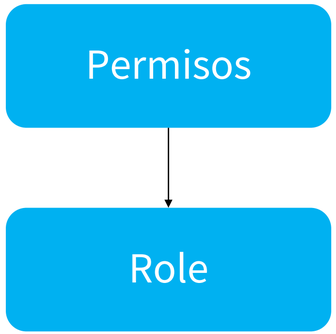


Autorización - RBAC

Ahora que ya sabemos como configurar usuarios y serviceAccount en el clúster, vamos a ver cómo funciona el sistema de permisos de Kubernetes.

El sistema de autorización de Kubernetes es un sistema basado en roles (RBAC). No se asignan permisos directamente a usuarios o serviceAccounts, sino que primero se definen roles que implementan distintos niveles de permisos, para luego asignar usuarios a cada rol.

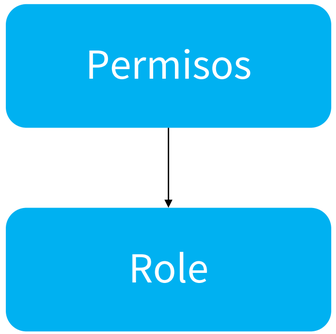
RBAC es el método estándar para definir permisos en muchos sistemas porque permite una gran flexibilidad y sobre todo poder reaprovechar roles para distintos usuarios, más sencillos de gestionar.



Autorización - RBAC

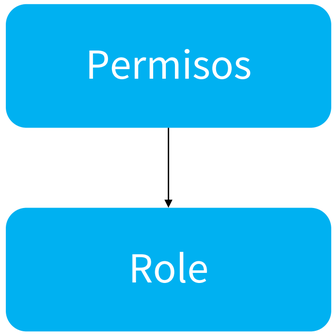
Veamos cómo es el YAML de un rol en Kubernetes:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: <Nombre del rol>
rules:
< Lista de permisos >
```



Autorización - RBAC

Es un objeto más de Kubernetes, de la API rbac.authorization.k8s.io/v1, y que sólo contiene un campo llamado rules. Dentro de rules va una lista con todos los permisos que tendrá ese rol asignados.



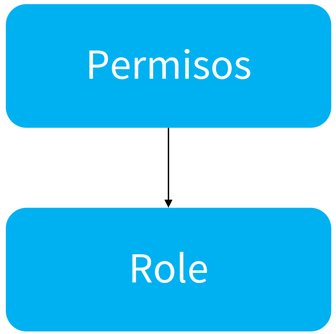
Cada elemento de la lista rules tendrá el siguiente formato:

```
- apiGroups: [ ]
  resources: [ ]
  verbs: [ ]
```

Autorización - RBAC

Cada permiso es para uno o varios recursos en concreto (pods, deployments, services, configMaps, etc.). Y para esos recursos se permiten una o varias acciones o verbos (get, list, update, create, patch, delete, watch).

apiGroups es la lista de APIs en la que se pueden encontrar los recursos. Por ejemplo, apps para los Deployments, DaemonSets y StatefulSets, batch para los jobs, networking.k8s.io para los Ingress, etc. Un elemeto vacío (sólo con las comillas), representará el API core de Kubernetes, v1, en la que se encuentran objetos como pods, services, configMaps, secrets, PVs, PVCs etc.

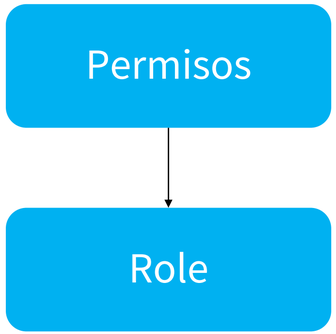


Autorización - RBAC

Puedes consultar la API de cualquier objeto en la referencia de la API de Kubernetes:

https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.18/#ingress-v1beta1-networking-k8s-io

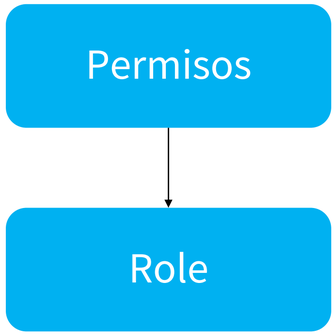
La ventaja de usar APIs disintas para los distintos tipos de objetos es que es un sistema fácilmente extensible. Aunque no lo veremos en este curso, es muy sencillo crear tus propias APIs y objetos para Kubernetes. Así, si has desplegado en tu clúster una herramienta que gestiona backups por ejemplo, esta tendrá su propio set de objetos, y tú puedes utilizar exactamente las mismas herramientas para dar permisos sobre ellos.



Autorización - RBAC

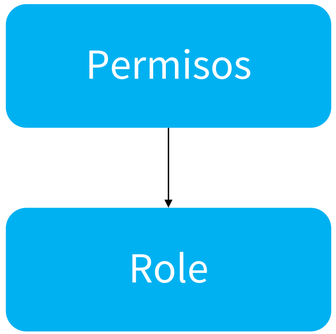
Veamos un ejemplo de rol real:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "watch", "list"]
```



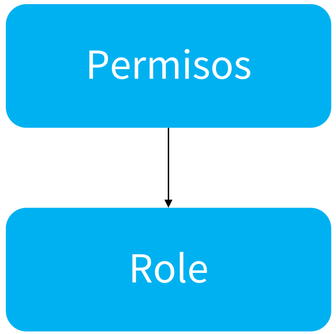
Autorización - RBAC

apiGroups es sólo un campo vacio para indicar la API Core V1. Los objetos sobre los que damos permisos son sólo los pods, y damos permiso para que puedan listar, obetener y observar los pods.



Autorización - RBAC

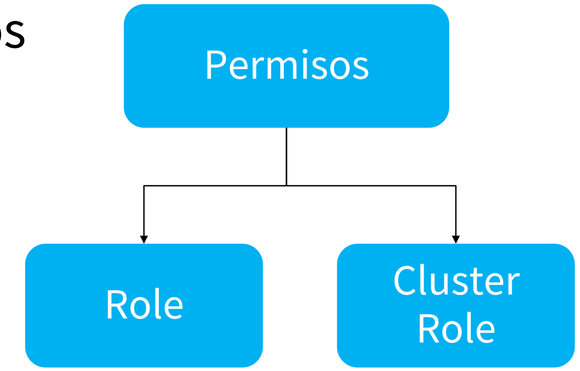
Otro ejemplo que da permisos sobre varios tipos de cargas de trabajo:




```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: workload-admin
rules:
- apiGroups: ["batch"]
  resources: ["jobs"]
  verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
- apiGroups: ["apps"]
  resources: ["deployments", "statefulsets", "daemonsets"]
  verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
```

Autorización - RBAC

Los roles siempre están limitados al namespace en el que se crean. Si queremos dar permisos para objetos en cualquier namespace, utilizamos el objeto ClusterRole:

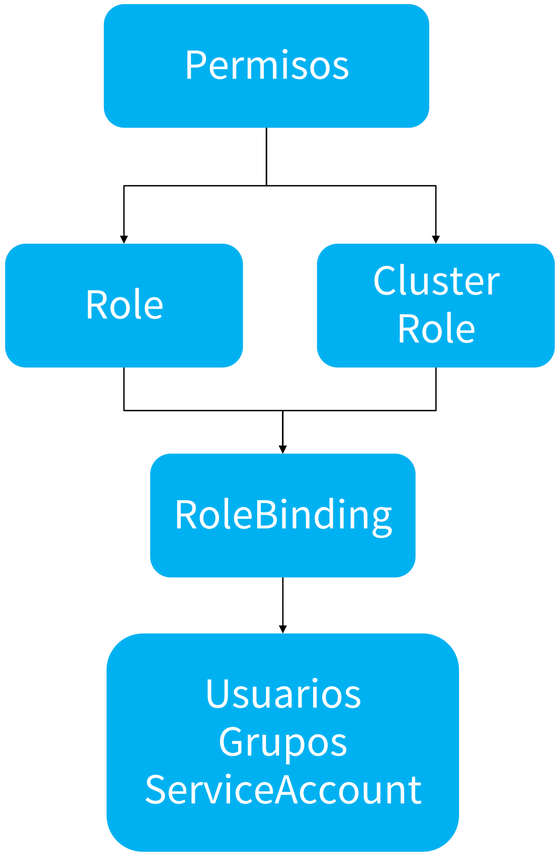



```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "watch", "list “]
```

Autorización - RBAC

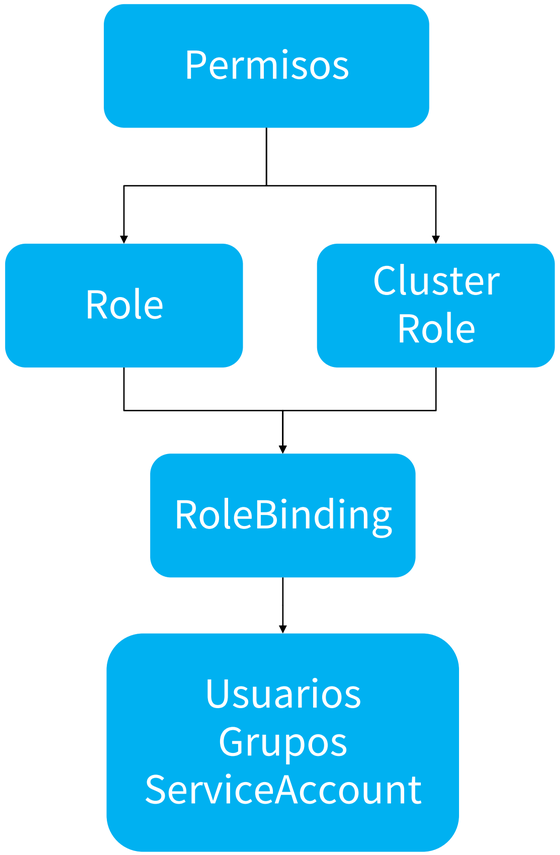
El objeto ClusterRole no tiene namespace.

Ahora que hemos creado algunos roles, vamos a ver cómo asignarlos a distintos usuarios. Esto lo haremos con el objeto RoleBinding.



Autorización - RBAC

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods-robert
subjects:
- kind: User
  name: robert
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

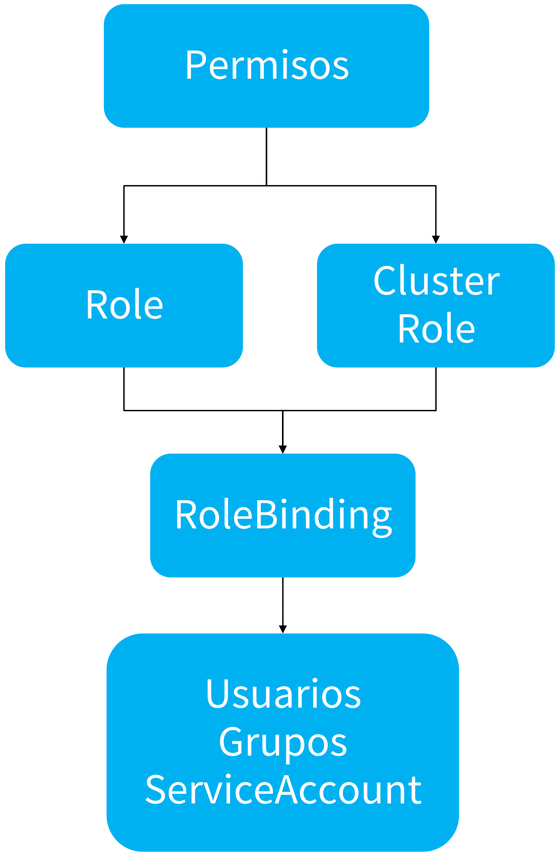


Autorización - RBAC

Un roleBinding tiene una lista de 'sujetos' y una referencia a un role o un clusterRole.

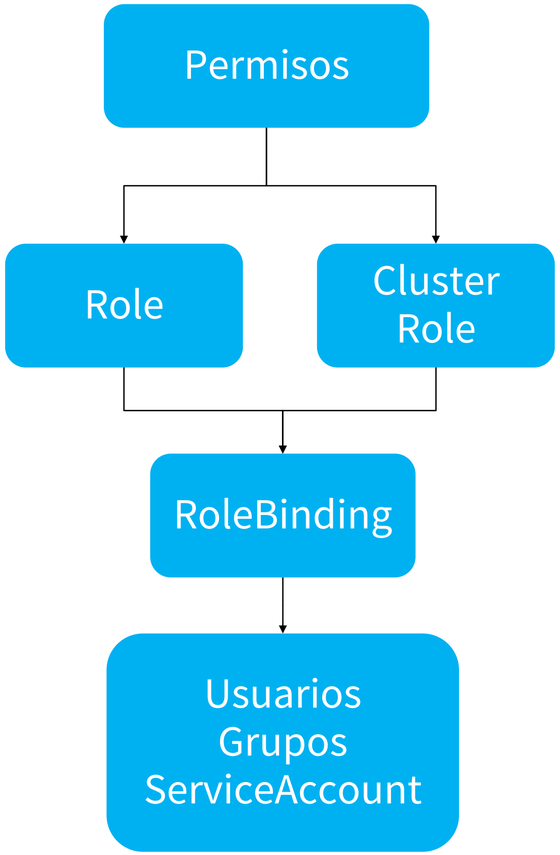
En este caso le estamos dando permisos sólo al usuario robert (este campo es case sensitive).

Podemos darle permisos también a un grupo de usuarios:



Autorización - RBAC

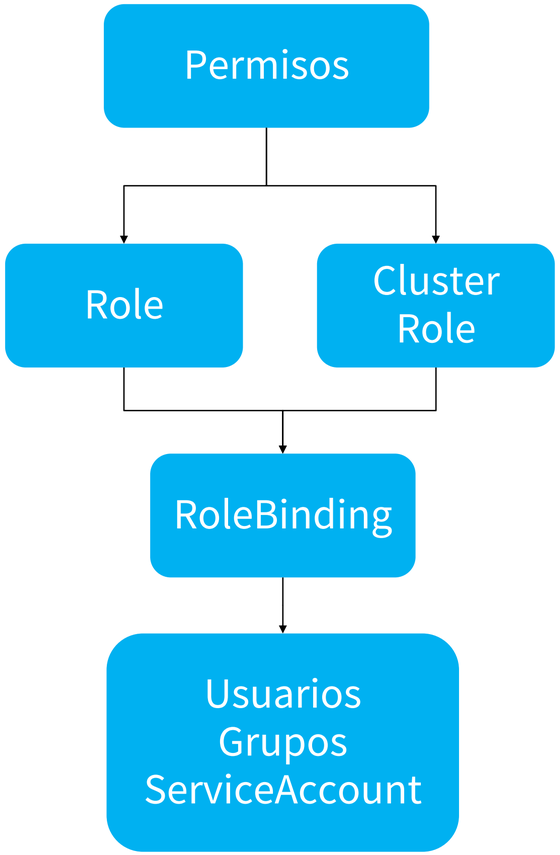
```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods-developers
subjects:
- kind: Group
  name: developers
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```



Autorización - RBAC

Y a una serviceAccount:

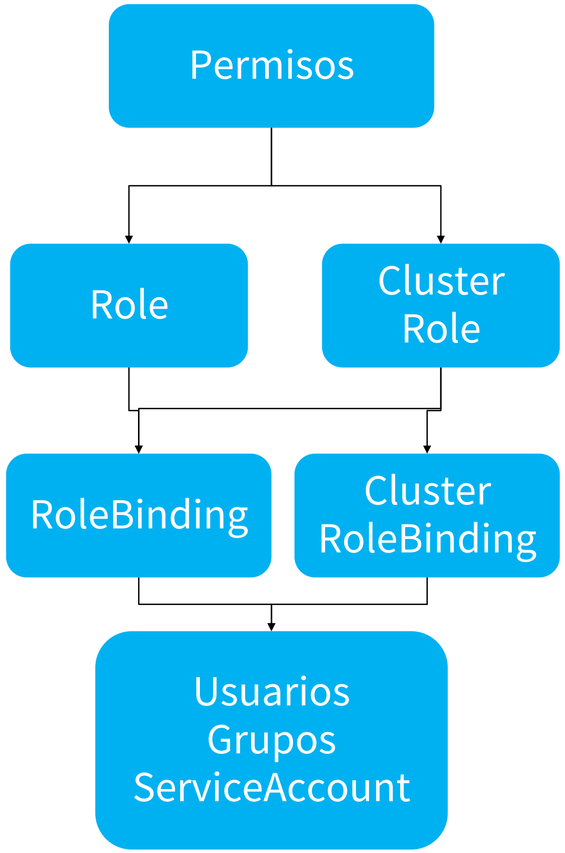
```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods-service-account
subjects:
- kind: ServiceAccount
  name: my-credentials
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```



Autorización - RBAC

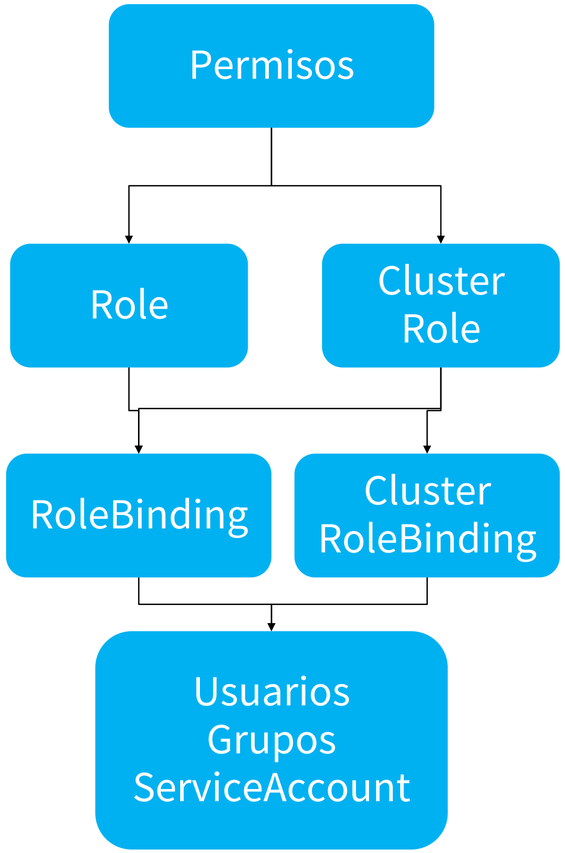
Los RoleBindings también están limitados al namespace al que pertenecen.

Aunque estemos asignando un ClusterRole, sólo darán permisos para los objetos que estén en el namespace. Para asignar permisos en todo el clúster usaremos ClusterRoleBinding



Autorización - RBAC

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: read-pods-developers
subjects:
- kind: Group
  name: developers
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```



¡Fin del módulo!

En este módulo hemos aprendido cómo gestionar la autenticación y autorización de Kubernetes.

- Hemos visto distintos sistemas de autenticación que soporta Kubernetes, y cómo utilizarlos en nuestro archivo kubeconfig para conectarnos al clúster
- Aprendido qué son serviceAccounts, su utilidad y cómo crearlas y montarlas en los pod automáticamente
- Conocido el sistema de permisos RBAC
- Creado Roles y ClusterRoles con permisos para distintos recursos y acciones
- Creado RoleBindings y ClusterRoleBindings para asignar usuarios y grupos a roles

Extras

Páginas de la documentación oficial:

Autenticación
ServiceAccounts
RBAC

¡Gracias!